DIDIT | 개발자 회의 플랫폼

개요	Did-It은 개발팀을 위한 협업 올인원 플랫폼으로, GitHub 이슈 관리·WebRTC 음성 회의·실시간 채팅·AI 기반 회의 요약을 하나의 워크스페이스에서 제공합니다.
기간	2026년 1월 5일 → 2026년 2월 9일
기술	Axios OpenVidu React SSE TailwindCSS Vite Zustand
팀원 구성	**FE(2)** , BE/INFRA(2), AI(2)
FE 기여도	35
AI 기여도	30
BE 기여도	35
역할 기여도	50

프로젝트 개요

개발팀이 프로젝트 진행 중 흩어진 협업 정보를 한 곳에서 관리할 수 있도록 **GitHub 이슈, 채팅, 음성 회의, AI 회의 요약을 통합한 협업 플랫폼**을 개발했습니다.

Did-It은 GitHub 저장소와 프로젝트 워크스페이스를 연결하고, 팀원이 이슈를 등록/관리하며, 채널 기반 채팅과 회의를 통해 논의한 내용을 회의록으로 남길 수 있는 서비스입니다. 특히 회의 녹음 이후 AI가 회의 내용을 요약해주어 팀원이 회의 후 해야 할 일을 빠르게 파악할 수 있도록 돕습니다.

프로젝트 설명

> 💡 **개발자들**만을 위한 회의 플랫폼, 어디 없을까?

개발 프로젝트에서는 이슈, 회의, 채팅, 회의록이 서로 다른 도구에 흩어지는 경우가 많습니다. 이로 인해 팀원은 현재 어떤 이슈가 중요한지, 회의에서 어떤 결정이 내려졌는지, 어떤 채널에서 논의가 이어졌는지 반복해서 확인해야 합니다.

Did-It은 이러한 불편을 줄이기 위해 프로젝트 단위의 워크스페이스 안에서 협업 흐름을 통합했습니다.

1. GitHub OAuth로 로그인하고 프로젝트 워크스페이스를 생성할 수 있습니다.
2. GitHub 저장소를 연결해 이슈를 관리하고, AI 기반 우선순위 분석 결과를 확인할 수 있습니다.
3. 채널별 실시간 채팅과 WebRTC 음성 회의를 통해 논의를 진행할 수 있습니다.
4. 회의 녹음 및 AI 요약을 통해 회의록을 자동 생성하고 관리할 수 있습니다.

프로젝트 의도를 위해 제공하는 핵심 기능은 아래와 같습니다.

- GitHub OAuth 기반 로그인 및 사용자 정보 관리
- 프로젝트 생성, 초대 링크 참여, 멤버 관리, 팀장 위임
- GitHub 저장소 연동 및 이슈 CRUD
- AI 기반 이슈 우선순위 분석
- SSE 기반 실시간 채팅 및 미디어 이벤트 동기화
- OpenVidu 기반 WebRTC 음성 회의, 화면 공유, 녹음 상태 제어
- AI 회의록 생성, 수정, 삭제, PDF 내보내기

실제 서비스 화면

담당 기능 개발

> 💡 **프론트엔드 화면/상태관리/실시간 기능/API 연동 중심**

1. React 라우팅 및 레이아웃 구조 설계

🤔 도입 배경 및 고민

- 서비스에는 공개 페이지, 로그인 필요 페이지, 프로젝트 워크스페이스, 회의실처럼 성격이 다른 화면이 섞여 있었습니다. 특히 워크스페이스 내부에서는 사이드바와 채팅 패널은 유지하면서 라운지/회의실/회의록 화면만 바뀌어야 했습니다.

💡 해결책 및 기술적 의사결정

- **인증 라우트 분리:** `PrivateRoute`를 두어 로그인 상태 확인 후 보호 페이지 접근을 제어했습니다. 앱 시작 시 `checkLoginStatus()`를 호출해 세션 기반 인증 상태를 먼저 복원하도록 구성했습니다.
- **중첩 레이아웃 설계:** `BaseLayout`, `MainLayout`, `WorkspaceLayout`을 분리하고, `/workspace/:projectId` 하위에 라운지와 회의실을 중첩시켰습니다. 이를 통해 프로젝트 사이드바, 채널 사이드바, 채팅 패널이 화면 전환 중에도 유지되도록 했습니다.

```
<Route element={<PrivateRoute />}>
  <Route element={<MainLayout />}>
    <Route path="/mypage" element={<MyPage />} />
  </Route>

  <Route path="/workspace/:projectId" element={<WorkspaceLa
yout />}>
    <Route index element={<Lounge />} />
    <Route path="meeting/:channelId" element={<MeetingRoom
/>} />
  </Route>
</Route>
```

---

2. Zustand 기반 도메인 상태 관리

🤔 도입 배경 및 고민

- 프로젝트, 이슈, 회의, 채팅, 사용자 정보가 여러 페이지와 컴포넌트에서 동시에 사용되기 때문에 props 전달만으로는 상태 흐름이 복잡해졌습니다. 또한 채팅/회의처럼 외부 연결 상태를 공유해야 하는 기능은 중앙 관리가 필요했습니다.

💡 해결책 및 기술적 의사결정

- **도메인별 Store 분리:** `useProjectStore`, `useLoungeStore`, `useMeetingStore`, `useChatStore`, `useAuthStore`로 상태 책임을 나누었습니다.

- **API 응답 정규화:** 이슈 목록, 프로젝트 목록처럼 API 응답 형태가 화면 요구사항과 다른 데이터는 store 계층에서 정규화해 컴포넌트가 렌더링에 집중할 수 있게 했습니다.
- **병렬 데이터 조회:** 프로젝트 목록을 가져온 뒤 프로젝트별 참여자 수를 `Promise.all`로 병렬 조회해 대시보드 렌더링 지연을 줄였습니다.

```
const validProjectIds = new Set(currentAllProjects.map((p)
=> p.id));
const filteredRecentProjects = recentProjects.filter(
  (recent) => recent.project && validProjectIds.has(recent.
project.id)
);
```

---

3. SSE 기반 실시간 채팅 및 이벤트 동기화

🤔 도입 배경 및 고민

- 채팅 메시지뿐 아니라 회의 참가자 입장/퇴장, 녹음 시작/종료, 미디어 오류 등 다양한 실시간 이벤트를 화면에 즉시 반영해야 했습니다. 또한 `WorkspaceLayout`과 `MeetingRoom` 양쪽에서 같은 채널의 이벤트를 구독하면서 EventSource가 중복 생성되는 문제가 있었습니다.

💡 해결책 및 기술적 의사결정

- **참조 카운트 기반 SSE 연결 관리:** 동일 채널에 이미 연결된 경우 실제 EventSource를 새로 만들지 않고 `connectionCount`만 증가시키도록 구현했습니다. 연결 해제 시 카운트를 감소시키고 0이 될 때만 `close()`를 호출했습니다.
- **이벤트 타입별 처리:** `chat_update`, `chat_delete`, `media_participant_joined`, `media_participant_left`, `media_recording_start`, `media_recording_stop`, `media_error` 이벤트를 구독해 채팅 UI와 회의 상태를 동기화했습니다.
- **자동 재연결:** SSE 오류 발생 시 현재 채널이 유지되고 있는지 확인한 뒤 3초 후 재연결하도록 처리했습니다.

```
if (isConnected && connectedChannelId === channelId && even
tSource) {
  set((state) => ({ connectionCount: state.connectionCount
+ 1 }));
  return;
}
```

---

4. OpenVidu 기반 WebRTC 회의 UX 구현

🤔 도입 배경 및 고민

- 회의실에서는 음성 회의, 화면 공유, 발화 감지, 참가자 목록, 녹음 상태가 동시에 변합니다. 사용자가 회의 중 화면 공유를 시작하거나 종료해도 오디오 퍼블리셔가 안정적으로 복구되어야 했고, 연결 종료 시 미디어 트랙과 세션 상태를 정리해야 했습니다.

💡 해결책 및 기술적 의사결정

- **OpenVidu 세션 훅 분리:** `useOpenVidu` 훅에 세션 참가, 마이크 토글, 스피커 토글, 화면 공유 시작/종료, 정리 로직을 모았습니다.
- **화면 공유와 오디오 복구 처리:** 화면 공유 시작 시 기존 오디오 publisher를 unpublish하고 screen publisher를 publish했습니다. 화면 공유 종료 후에는 오디오 전용 publisher를 다시 생성해 음성 회의가 이어지도록 했습니다.
- **발화 감지 UI:** OpenVidu의 `publisherStartSpeaking`, `publisherStopSpeaking` 이벤트를 구독해 참가자별 발화 상태를 store에 반영했습니다.

```
OV.setAdvancedConfiguration({
  publisherSpeakingEventsOptions: {
    interval: 100,
    threshold: -50,
  },
});
```

---

5. 공통 UI 컴포넌트 및 API 레이어 정리

🤔 도입 배경 및 고민

- 프로젝트 생성, 멤버 관리, 이슈 수정, 프로필 편집처럼 모달과 버튼, 입력 컴포넌트가 반복되었습니다. UI의 통일성과 개발 환경에서의 간편함을 위해 공통 컴포넌트들은 서비스 컨셉에 맞추어 통일을 시킬 필요가 있었습니다. 또한, API 호출도 도메인별로 흩어지면 유지보수가 어려웠습니다.

💡 해결책 및 기술적 의사결정

- **공통 컴포넌트 분리:** `Button`, `IconButton`, `Input`, `Modal`, `ConfirmModal`, `Toast`, `Badge`, `Avatar`, `Card`, `Checkbox`, `Divider`를 `components/common`에 정리했습니다.
- **도메인 API 모듈화:** `auth.js`, `project.js`, `meeting.js`, `lounge.js`, `features.js`로 API 레이어를 나누고, `axios.js`에서 공통 baseURL, credentials, interceptor 설정을 관리했습니다.

- **Mock 모드 지원:** `VITE_USE_MOCK=true` 환경변수로 백엔드 없이 프론트 개발과 UI 검증이 가능하도록 분기했습니다.

트러블슈팅

✨ 문제 1. Optimistic UI와 SSE 메시지 중복 표시

문제 배경

채팅 전송 시 사용자가 지연을 느끼지 않도록 임시 메시지를 먼저 렌더링했습니다. 그런데 서버에서 SSE로 같은 메시지가 다시 도착하면서 내 메시지가 두 번 표시되는 문제가 발생했습니다. 특히 SSE payload에는 실제 DB ID가 없는 경우가 있어 임시 메시지를 안정적으로 교체하기 어려웠습니다.

💡 해결 방법

내가 보낸 메시지는 SSE 수신 처리에서 제외하고, `sendMessage` API 성공 후 `fetchRecentChats`를 silent 옵션으로 호출해 서버의 실제 메시지 목록으로 교체했습니다.

```
if (newMessage.isMine) {
  return state;
}

await chatApi.createChat(channelId, { content });
await get().fetchRecentChats(channelId, user, { silent: tru
e });
```

📝 알게된 점

실시간 이벤트와 Optimistic UI를 함께 사용할 때는 이벤트 스트림을 그대로 화면에 누적하기보다, 어떤 데이터를 단일 진실 공급원으로 삼을지 먼저 정해야 한다는 점을 배웠습니다.

✨ 문제 2. SSE 연결 중복 생성

문제 배경

워크스페이스 레이아웃과 회의실 컴포넌트가 같은 채널 이벤트를 동시에 필요로 하면서 EventSource가 중복 생성되었습니다. 그 결과 메시지와 미디어 이벤트가 중복 수신되는 문제가 있었습니다.

💡 해결 방법

`useChatStore`에서 현재 연결된 채널과 EventSource를 전역으로 관리하고, 같은 채널에 이미 연결되어 있으면 참조 카운트만 증가시키도록 구현했습니다.

```
const nextCount = Math.max(0, connectionCount - 1);

if (nextCount === 0) {
  eventSource.close();
  set({ eventSource: null, isConnected: false, connectionCo
unt: 0 });
} else {
  set({ connectionCount: nextCount });
}
```

📝 **알게된 점**

여러 컴포넌트가 하나의 외부 연결을 공유할 때는 컴포넌트 생명주기에만 맡기면 중복 연결이 쉽게 발생합니다. 연결 자체를 store에서 싱글톤처럼 관리하고, 구독자 수를 별도로 추적하는 방식이 효과적이었습니다.

향후 개선 계획

1. 실시간 연결 안정성 강화

- **현재의 한계:**
  - SSE 오류 발생 시 단순 3초 재연결 방식으로 복구하고 있어, 네트워크 상태가 불안정한 경우 재연결 시도가 과도하게 반복될 수 있습니다.
- **개선 방안:**
  - exponential backoff를 적용하고, 사용자가 채널을 떠난 경우 예약된 재연결을 취소하는 방식으로 연결 안정성을 높일 수 있습니다.

2. 프론트엔드 테스트 도입

- **현재의 한계:**
  - 채팅 store, OpenVidu 훅, 프로젝트 필터링처럼 상태 변화가 중요한 로직에 대한 자동화 테스트가 부족합니다.
- **개선 방안:**

- Vitest와 React Testing Library를 도입해 store 단위 테스트, 라우팅 보호 테스트, 주요 컴포넌트 렌더링 테스트를 구성할 수 있습니다.

3. 회의 UX 고도화

- **현재의 한계:**
  - 회의 상태와 화면 공유 상태는 제공하지만, 회의 중 네트워크 품질이나 장치 권한 문제를 사용자가 더 쉽게 이해할 수 있는 안내가 부족합니다.
- **개선 방안:**
  - 마이크 권한, 화면 공유 권한, 연결 품질, 녹음/요약 진행 상태를 더 세분화해 안내하고, 실패 상황별 복구 액션을 UI에 제공할 수 있습니다.